

Spring Boot — Complete Detailed Notes

1. What is Spring Boot?

Spring Boot is built on top of the Spring Framework. It removes all the manual configuration that normal Spring requires. You don't write XML files, you don't set up a web server manually, you don't configure a DispatcherServlet — Spring Boot does all of this automatically.

The three core ideas of Spring Boot:

- **Auto-configuration** — Spring Boot automatically configures your application based on what dependencies you have added
 - **Embedded server** — Tomcat is embedded inside your app. You don't deploy a WAR file to an external server. You just run the JAR file.
 - **Starter dependencies** — Instead of adding 10 individual dependencies, you add one starter (like `spring-boot-starter-web`) and Spring Boot pulls everything needed automatically
-

2. Spring Boot vs Spring Framework

Feature	Spring Framework	Spring Boot
Configuration	Manual XML or Java config	Auto-configured
Server setup	External Tomcat needed	Embedded Tomcat included
Dependency management	Add each dependency manually	Starter dependencies handle it
Startup time	More setup required	Run directly with <code>main()</code>
Production readiness	Manual setup	Actuator included

3. @SpringBootApplication

This is the most important annotation in Spring Boot. It is placed on the main class.

```
java
```

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

`@SpringBootApplication` is a combination of three annotations:

Annotation	What it does
<code>@SpringBootConfiguration</code>	Same as <code>@Configuration</code> — marks this class as a config class
<code>@EnableAutoConfiguration</code>	Tells Spring Boot to auto-configure beans based on classpath
<code>@ComponentScan</code>	Scans the current package and all sub-packages for beans

Important: All your controllers, services, repositories must be in the **same package or sub-packages** as the main class. Otherwise `@ComponentScan` won't find them.

4. How Auto-Configuration Works

When you add `spring-boot-starter-web` to `pom.xml`, Spring Boot sees that you have web-related classes on the classpath. It then automatically:

- Configures a `DispatcherServlet`
- Sets up embedded Tomcat on port 8080
- Configures Jackson for JSON conversion
- Sets up error handling

This is done through `@EnableAutoConfiguration`. Spring Boot reads a file called `spring.factories` (or `AutoConfiguration.imports` in newer versions) which contains a list of auto-configuration classes. Spring Boot runs through this list and applies the ones that are relevant based on what's on your classpath.

You can see what was auto-configured by adding this to `application.properties`:

```
debug=true
```

5. application.properties / application.yml

This file is where you configure your Spring Boot application. It lives in `src/main/resources`.

application.properties format

```
properties

# Server config
server.port=8081
server.servlet.context-path=/api

# Database config
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA config
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# Custom property
app.name=MyApplication
app.max-users=100
```

application.yml format (same thing, different format)

```
yaml

server:
  port: 8081
  servlet:
    context-path: /api

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: root
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

spring.jpa.hibernate.ddl-auto values

Value	Behavior
<code>create</code>	Drops and creates tables every time app starts
<code>create-drop</code>	Creates tables on start, drops on shutdown
<code>update</code>	Updates table structure without dropping data
<code>validate</code>	Validates schema, throws error if mismatch
<code>none</code>	Does nothing

Use `update` in development, `validate` or `none` in production.

6. Spring Boot Starter Dependencies

Starters are pre-packaged dependency bundles. You add one starter and get everything needed for that functionality.

Starter	What it includes
<code>spring-boot-starter-web</code>	Spring MVC, Tomcat, Jackson
<code>spring-boot-starter-data-jpa</code>	Hibernate, Spring Data JPA
<code>spring-boot-starter-security</code>	Spring Security
<code>spring-boot-starter-test</code>	JUnit, Mockito, Spring Test
<code>spring-boot-starter-aop</code>	Spring AOP, AspectJ
<code>spring-boot-starter-validation</code>	Bean Validation (Hibernate Validator)

```
xml
```

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

7. Spring MVC — How a Request is Handled

This is the flow of every HTTP request in a Spring Boot web application:

```
Client sends HTTP request
    ↓
Embedded Tomcat receives it
    ↓
DispatcherServlet receives the request (Front Controller)
    ↓
HandlerMapping finds which Controller method matches the URL
    ↓
Controller method executes, calls Service layer
    ↓
Controller returns data (or view name)
    ↓
If @RestController → Jackson converts object to JSON → sent to client
If @Controller    → ViewResolver finds the view (JSP/Thymeleaf) → rendered → sent
to client
```

DispatcherServlet is the front controller. Every request goes through it first. It then delegates to the right controller.

8. REST Controller Annotations

@RestController

Marks a class as a REST controller. Every method automatically returns data in the response body (JSON by default). Combination of `@Controller` + `@ResponseBody`.

```
java

@RestController
@RequestMapping("/api/users")
public class UserController {
    // all methods here return JSON
}
```

@RequestMapping

Maps HTTP requests to controller methods. Can be applied at class level (base URL) and method level (specific URL).

```
java

@RestController
@RequestMapping("/api/users") // base URL for all methods in this class
public class UserController {

    @RequestMapping(value = "/all", method = RequestMethod.GET)
    public List<User> getAllUsers() { ... }
}
```

@GetMapping

Shortcut for `@RequestMapping(method = RequestMethod.GET)`. Handles GET requests. Used to fetch data.

```
java
```

```
@GetMapping("/all")
public List<User> getAllUsers() {
    return userService.getAllUsers();
}

@GetMapping("/{id}")
public User getUserById(@PathVariable int id) {
    return userService.getUserById(id);
}
```

@PostMapping

Handles POST requests. Used to create new data.

```
java

@PostMapping("/create")
public User createUser(@RequestBody User user) {
    return userService.createUser(user);
}
```

@PutMapping

Handles PUT requests. Used to update existing data (full update).

```
java

@PutMapping("/{id}")
public User updateUser(@PathVariable int id, @RequestBody User user) {
    return userService.updateUser(id, user);
}
```

@PatchMapping

Handles PATCH requests. Used for partial update (update only specific fields).

```
java
```

```
@PatchMapping("/{id}/email")
public User updateEmail(@PathVariable int id, @RequestBody String email) {
    return userService.updateEmail(id, email);
}
```

@DeleteMapping

Handles DELETE requests. Used to delete data.

```
java

@DeleteMapping("/{id}")
public void deleteUser(@PathVariable int id) {
    userService.deleteUser(id);
}
```

9. Method Parameter Annotations

@PathVariable

Extracts a value from the URL path.

```
java

// URL: GET /api/users/5
@GetMapping("/{id}")
public User getUser(@PathVariable int id) {
    // id = 5
    return userService.findById(id);
}

// Multiple path variables
// URL: GET /api/users/5/orders/10
@GetMapping("/{userId}/orders/{orderId}")
public Order getOrder(@PathVariable int userId, @PathVariable int orderId) {
    return orderService.findOrder(userId, orderId);
}
```

@RequestParam

Extracts a value from the URL query string (after (?)).

```
java

// URL: GET /api/users?name=John&age=25
@GetMapping
public List<User> searchUsers(
    @RequestParam String name,
    @RequestParam int age) {
    return userService.search(name, age);
}

// Optional param with default value
// URL: GET /api/users or GET /api/users?page=2
@GetMapping
public List<User> getUsers(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size) {
    return userService.getUsers(page, size);
}
```

@RequestBody

Reads the HTTP request body and converts it (deserializes JSON) into a Java object using Jackson.

```
java

// POST /api/users
// Request body: {"name": "John", "email": "john@gmail.com"}
@PostMapping
public User createUser(@RequestBody User user) {
    // user object is populated with data from request body
    return userService.save(user);
}
```

@RequestHeader

Extracts a value from the HTTP request header.

```
java
```

```
@GetMapping("/profile")
public User getProfile(@RequestHeader("Authorization") String token) {
    // reads Authorization header value
    return userService.getUserFromToken(token);
}
```

@ResponseStatus

Sets the HTTP response status code for a method.

```
java

@PostMapping
@ResponseStatus(HttpStatus.CREATED) // returns 201 instead of default 200
public User createUser(@RequestBody User user) {
    return userService.save(user);
}
```

10. ResponseEntity

`ResponseEntity` gives you full control over the HTTP response — status code, headers, and body.

```
java
```

```

@GetMapping("/{id}")
public ResponseEntity<User> getUser(@PathVariable int id) {
    User user = userService.findById(id);

    if (user == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(null);
    }

    return ResponseEntity.ok(user); // 200 OK with user in body
}

@PostMapping
public ResponseEntity<User> createUser(@RequestBody User user) {
    User saved = userService.save(user);
    return ResponseEntity
        .status(HttpStatus.CREATED) // 201
        .header("Custom-Header", "value")
        .body(saved);
}

```

Common `ResponseEntity` shortcuts:

```

java

ResponseEntity.ok(data) // 200
ResponseEntity.created(URI).body(data) // 201
ResponseEntity.noContent().build() // 204
ResponseEntity.badRequest().body(error) // 400
ResponseEntity.notFound().build() // 404
ResponseEntity.status(HttpStatus.FORBIDDEN) // 403

```

11. Spring Boot with JPA

What is JPA?

JPA (Java Persistence API) is a specification for ORM (Object Relational Mapping). It lets you map Java classes to database tables and perform database operations using Java objects instead of writing SQL.

Hibernate is the most popular implementation of JPA. Spring Boot uses Hibernate by default.

@Entity

Marks a Java class as a database table. Each field maps to a column.

```
java

@Entity
@Table(name = "users") // maps to "users" table in DB, optional if class name matches
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // auto-increment
    private int id;

    @Column(name = "user_name", nullable = false, length = 50)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // getters and setters
}
```

JPA Annotations

Annotation	Purpose
<code>@Entity</code>	Marks class as a DB table
<code>@Table(name="")</code>	Specifies table name
<code>@Id</code>	Marks primary key field
<code>@GeneratedValue</code>	Auto-generates primary key value
<code>@Column</code>	Customizes column mapping
<code>@Transient</code>	Field is NOT mapped to any column
<code>@Lob</code>	Maps large objects (text/blob)
<code>@Enumerated</code>	Maps enum to DB column

GenerationType strategies

Strategy	Behavior
IDENTITY	DB auto-increments (MySQL)
SEQUENCE	Uses DB sequence (PostgreSQL, Oracle)
AUTO	Spring picks strategy automatically
TABLE	Uses a separate table to generate IDs

@Repository — Spring Data JPA

`JpaRepository` provides all basic CRUD operations out of the box. You just extend it — no implementation needed.

```
java

@Repository
public interface UserRepository extends JpaRepository<User, Integer> {
    // JpaRepository<EntityClass, PrimaryKeyType>
    // All these methods are available automatically:
    // save(), findById(), findAll(), deleteById(), count(), existsById()
}
```

Custom Query Methods (Derived Queries)

Spring Data JPA can generate queries just from the method name.

```
java
```

@Repository

```
public interface UserRepository extends JpaRepository<User, Integer> {

    // SELECT * FROM users WHERE name = ?
    List<User> findByName(String name);

    // SELECT * FROM users WHERE email = ?
    Optional<User> findByEmail(String email);

    // SELECT * FROM users WHERE name = ? AND email = ?
    List<User> findByNameAndEmail(String name, String email);

    // SELECT * FROM users WHERE name LIKE '%?%'
    List<User> findByNameContaining(String keyword);

    // SELECT * FROM users WHERE age > ?
    List<User> findByAgeGreaterThan(int age);

    // SELECT * FROM users ORDER BY name ASC
    List<User> findAllOrderByNameAsc();

    // SELECT COUNT(*) FROM users WHERE name = ?
    long countByName(String name);

    // DELETE FROM users WHERE name = ?
    void deleteByName(String name);
}
```

@Query — Custom JPQL or Native SQL

When derived query methods are not enough, write your own query.

```
java
```

@Repository

```
public interface UserRepository extends JpaRepository<User, Integer> {

    // JPQL query (uses class name and field name, not table/column name)
    @Query("SELECT u FROM User u WHERE u.email = :email")
    Optional<User> findUserByEmail(@Param("email") String email);

    // Native SQL query (uses actual table and column names)
    @Query(value = "SELECT * FROM users WHERE user_name = :name", nativeQuery = true)
    List<User> findByNameNative(@Param("name") String name);

    // Update query needs @Modifying and @Transactional
    @Modifying
    @Transactional
    @Query("UPDATE User u SET u.email = :email WHERE u.id = :id")
    int updateEmail(@Param("id") int id, @Param("email") String email);
}
```

Complete CRUD Service Example

java

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    // CREATE
    public User createUser(User user) {
        return userRepository.save(user);
    }

    // READ ALL
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    // READ BY ID
    public User getUserById(int id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("User not found with id: " + id));
    }

    // UPDATE
    public User updateUser(int id, User updatedUser) {
        User existing = getUserById(id);
        existing.setName(updatedUser.getName());
        existing.setEmail(updatedUser.getEmail());
        return userRepository.save(existing); // save() updates if ID exists
    }

    // DELETE
    public void deleteUser(int id) {
        userRepository.deleteById(id);
    }
}
```

Complete REST Controller Example

```
java
```



```

@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
        return ResponseEntity.ok(userService.getAllUsers());
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(@PathVariable int id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        User saved = userService.createUser(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @PutMapping("/{id}")
    public ResponseEntity<User> updateUser(
        @PathVariable int id,
        @RequestBody User user) {
        return ResponseEntity.ok(userService.updateUser(id, user));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable int id) {
        userService.deleteUser(id);
        return ResponseEntity.noContent().build();
    }
}

```

12. Exception Handling

@ExceptionHandler

Handles exceptions thrown by controller methods in the same controller.

```
java
```

```

@RestController
@RequestMapping("/api/users")
public class UserController {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<String> handleRuntimeException(RuntimeException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }
}

```

@ControllerAdvice / @RestControllerAdvice

Global exception handler — handles exceptions from ALL controllers in one place.

```

java

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<String> handleRuntimeException(RuntimeException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGenericException(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Something went wrong: " + ex.getMessage());
    }
}

```

13. Spring Boot Actuator

Actuator adds production-ready monitoring endpoints to your app automatically.

```

xml

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

```

properties

`management.endpoints.web.exposure.include=*`

Endpoint	What it shows
<code>/actuator/health</code>	App health status
<code>/actuator/info</code>	App info
<code>/actuator/beans</code>	All Spring beans
<code>/actuator/env</code>	Environment properties
<code>/actuator/metrics</code>	App metrics

14. Interview Questions

Q: What is Spring Boot and how is it different from Spring? Spring Framework requires manual configuration — XML files, bean definitions, server setup. Spring Boot removes all of this with auto-configuration, embedded server, and starter dependencies. Spring Boot is opinionated — it makes default decisions for you so you can start quickly.

Q: What does @SpringBootApplication do? It is a combination of `@SpringBootConfiguration`, `@EnableAutoConfiguration`, and `@ComponentScan`. It marks the main class, enables auto-configuration, and scans the current package and sub-packages for components.

Q: What is auto-configuration in Spring Boot? Spring Boot checks what dependencies are on the classpath and automatically configures beans accordingly. For example, if `spring-boot-starter-web` is present, Spring Boot auto-configures a DispatcherServlet and embedded Tomcat without you writing any configuration.

Q: What is the difference between @Controller and @RestController? `@Controller` returns a view name which is resolved by a ViewResolver (used for JSP, Thymeleaf). `@RestController` is `@Controller + @ResponseBody` — every method automatically writes the return value directly to the HTTP response body as JSON.

Q: What is the difference between @PathVariable and @RequestParam? `@PathVariable` extracts values from the URL path — `/users/5` where 5 is the path variable. `@RequestParam` extracts values from the query string — `/users?id=5` where id is the request param.

Q: What is the difference between PUT and PATCH? PUT replaces the entire resource with the new data. PATCH updates only the specified fields of the resource.

Q: What does `spring.jpa.hibernate.ddl-auto=update` do? It tells Hibernate to compare the entity classes with the existing database schema and update the schema (add columns, create tables) without dropping existing data.

Q: What is `ResponseEntity`? `ResponseEntity` is a Spring class that represents the entire HTTP response — status code, headers, and body. It gives you full control over what gets sent back to the client.

Q: What is the difference between `save()` and `saveAndFlush()` in `JpaRepository`? `save()` saves the entity and may batch the SQL until the transaction commits. `saveAndFlush()` immediately synchronizes the changes to the database within the current transaction.

15. Tricky Questions

Q: Can you have multiple `@SpringBootApplication` classes? Technically yes, but it causes conflicts because each one triggers component scanning and auto-configuration. You should have only one main class with `@SpringBootApplication`.

Q: If you add `@ComponentScan` on the main class with a different package, what happens? Spring Boot scans only the specified package and stops scanning the default package (where the main class lives). This means beans in the main class's package won't be discovered unless explicitly included.

Q: What is the difference between `@RestControllerAdvice` and `@ControllerAdvice`? `@ControllerAdvice` is for controllers that return views. `@RestControllerAdvice` adds `@ResponseBody` — exception handler methods automatically serialize their return value to JSON. Use `@RestControllerAdvice` for REST APIs.

Q: What happens if two auto-configuration classes try to configure the same bean? Spring Boot uses `@ConditionalOnMissingBean` in auto-configuration classes. If you define your own bean, Spring Boot detects it and skips its own auto-configuration for that bean. Your custom bean takes priority.

Q: Can `@Transactional` work on a private method? No. `@Transactional` works through Spring's proxy mechanism. Proxies can only intercept public methods. Putting `@Transactional` on a private method has no effect — the transaction never starts.

Q: What is the difference between `findById()` and `getOne()`? `findById()` immediately hits the database and returns an `Optional<T>`. `getOne()` (now `getById()`) returns a proxy and only hits the database when you access a field — lazy loading. If the entity doesn't exist, `getOne()` throws `EntityNotFoundException` when accessed.